

Poultry Advisory Agent — Complete System Design & Engineering Journey

BirdVet / Poultry Sam — a conversational AI poultry advisory agent for Nigerian semi-commercial farmers (200–2,000 birds). Status: design locked (RAG · Media · WhatsApp · Orchestrator). Scope: hackathon MVP toward production.

1. The problem

Nigerian poultry farming is two worlds. Village backyard flocks (5–50 birds, mostly women's, the majority of Nigeria's ~137M birds) and large industrial operations (10,000+). In between sits the **middle tier**: semi-commercial farmers running 200–2,000 birds. These are real businesses. Each cycle carries hundreds of thousands of naira of working capital, feed eating 45–68% of costs, and one disease outbreak can wipe an entire cycle.

Diseases are fast and unforgiving. Newcastle Disease alone has killed 55–80% of unvaccinated village flocks in bad years and can take 80–100% of a susceptible flock. Gumboro, fowlpox, and avian influenza follow the same pattern: symptoms appear, birds die within days, and treatment must begin on day one to matter.

But the system around the farmer is too slow. The closest veterinarian is effectively unreachable for most. The agro-vet store — the private drug and vaccine shop that actually serves this tier — is a ride away and the farmer does not know if they stock the right thing. And critically, **nothing about a local outbreak ever reaches authorities**. Outbreaks are discovered by the ministry weeks after the fact, if at all. Whole-region losses stay invisible until they become national crises.

The information gap is lethal because these diseases are *solvable in the field*: vaccination prevents Newcastle entirely, treatment exists for most others, and a one-day head-start is often the difference between a cycle and a catastrophe. The farmer does not lack willingness — they lack an affordable, instant, trustworthy advisor that lives inside the channel they already use: WhatsApp.

Core problem statement: *The disease is faster than the information. A farmer needs advice in minutes, from a channel they already use, in a language they actually speak — and the insight from one farm's outbreak should protect every farm nearby.*

2. The research journey — how we arrived at the shape

We did not start here. The original idea was broader, and research reshaped it five times.

Iteration 1 — generic livestock triage to vets. The first brief was a conversational agent that extracted case data and escalated to vets. Testing against the real market: **Doorcas Africa** already does web/WhatsApp AI chat for Nigerian livestock (disease reporting, vaccination reminders, ~1,000 users). DigiFarm (Safaricom, 1.6M farmers) does assistant chat; iCow does SMS livestock info in Kenya; Plantix does crop diagnosis. **Conclusion:** the "chatbot for farmers" wedge is occupied. What nobody does is

the layer above the chat — the structure, the decision, the handoff. We chose to be the skeleton under the conversation, not another conversation.

Iteration 2 — species pivot to poultry. Generic livestock thins the knowledge base. Poultry is where the economics concentrate: ~55% of Nigeria's livestock vaccine demand (~165M doses/yr, ~\$35M/yr opportunity), domestic production covers only ~25% of it, and Newcastle is the single highest-mortality, most-preventable problem in the sector. Agro-vet stores — the real rural distribution channel — already carry the vaccine a farmer needs. And a national initiative (FMLD + GALVmed ND Technical Working Group, 2026) is explicitly hunting for farmer-level disease detection. **Poultry became the species; the ministry became a stakeholder with a real need.**

Iteration 3 — the segment, and a correction on gender. The original brief implied "poor smallholder, likely a woman." Research corrected both assumptions:

- The tier that **suffers highest absolute financial loss and can afford to act** is the 200–2,000-bird semi-commercial middle tier — the dominant commercial farm size in Nigeria (Jos sample: 36% at 200–500, 25% at 501–1,000, 19% at 1,001–2,000). They already buy vaccines (near 100% vaccinate), mostly from vet/agro-vet shops (62.5% in one Ibadan study).
- Gender is *not* a persona axis. Studies split ~62–92% male operators in semi-commercial poultry; but women dominate daily flock care and are the earliest disease detectors, while men typically control purchase decisions. **The persona is the person responsible for the flock's health and its money** — sometimes identical, sometimes a carer reporting and a purchaser approving. The product must serve both hands on one phone.

Iteration 4 — economy of the "vet" door collapses. Escalating everything to a real vet is a cold-start dead end (and post-MVP). What actually works in this economy is a **severity-ranked decision tree**: resolve it safely myself → send a *pre-briefed referral* to an agro-vet store → escalate to a real vet only for serious/notifiable/high-value → report anonymised signals upward. The farmer pays nothing; agro-vet/vaccine/feed companies and the ministry pay.

Iteration 5 — the honest boss: the conversation must feel human. The farmer should never feel they are filling forms. The agent slips structured-data questions into a natural chat ("sorry to hear — how many have died so far?"), invisibly building the case card. **Natural conversation → structured case → decision → handoff.** That was the final lock.

3. The solution — one agent, four doors

A farmer sends a normal WhatsApp message — text, voice note, or photo — in Pidgin or English. The agent converses naturally, quietly extracting a structured case (species, flock size, symptoms, death count, vaccine history, feed, location). Each turn the orchestrator decides whether data is complete, asks what is missing conversationally, and routes to the right outcome:

1. **RESOLVE** — safe, instant husbandry/guidance inline. Never a diagnosis — guidance grounded in the knowledge base, with sources.
2. **SUPPLY** — a WhatsApp-forwardable *referral slip*: symptoms, suspected issue, and exactly what to ask for at the nearest agro-vet store (with a price hint). No store network required today — the farmer carries the handoff.

3. **ESCALATE** — post-MVP, real-vet dispatch with full case context. **Today, one safe rule:** notifiable/avian-influenza-class signs (sudden mass deaths, swollen comb/wattles, bleeding) → *"this needs a real vet on site today."* Unsupported, never silent.
4. **REPORT** — every interaction emits an anonymised outbreak signal (LGA, species, suspected issue) into the surveillance feed — the B2G story the ministry needs.

Revenue model: B2B first (agro-vet stock intel + qualified leads, vaccine/feed reach + compliance data), B2G second (outbreak early warning). Farmer pays nothing for the base service.

4. Engineering decisions — rationale and locks

Stack: TypeScript, React + Hono + Vercel AI SDK, Postgres + pgvector, R2.

- *Why TS/Hono over Next:* TS-heavy team, no server actions, no forced `/app/api` — plain routes, deployable anywhere; AI SDK's `useChat` / `streamText` + Zod tool schemas provide the agent layer.
- *Why not FastAPI/LangGraph:* TS-first team; porting the orchestrator to Python would mean building our own agent library — overkill for this scope. Python remains a future skill-bet, not this ship.

Orchestrator owns state, not the LLM. The greatest reliability leak is trusting the model to know what is missing. The orchestrator holds `session_state`, computes `missing()` in deterministic code, sends the LLM only `{filled, missing, query}` (tiny context = token-efficient), and the LLM returns `{delta, reply}`. Orchestrator merges, validates, persists. **LLM talks; orchestrator decides.**

Persistence: chunk-by-chunk sessions, atomic reports. Farmers abandon chats (60%+ will). Every turn UPSERTs the session state to Postgres — a partial conversation is never lost and still yields a surveillance signal even when incomplete. Only a fully validated case logs to the immutable `reports` table, keyed by `session_id` to stay idempotent against duplicate events.

RAG — designed to be trustworthy, not just clever. The central fear: *bad embedding* → *wrong output*. Three decisions contain it:

- *Page-stamp before the LLM ever sees text.* We paginate deterministically, inject `[PAGE n]` markers, then call the LLM once per document (`analyze_document`) to produce a region map. The LLM cannot be "off by a page" — it read the stamps. Headings are captured as rescue anchors.
- *Regions, not chapters-by-disease.* Textbooks mix diseases in one chapter and differential tables cross-reference everything. So metadata is **arrays**: a `chunk_group` carries `diseases[]`, `section_types[]`, `species[]`, and every chunk inherits its group's arrays. One map-call per book (~1 LLM call total); everything after is deterministic split + micro-chunking + embedding.
- *Retrieval is vector-core with two-stage sharpness and a starvation fallback.* Stage A (cold, disease unknown): pure vector top-k with a **diversity cap** (max 2 chunks/group — prevents one section flooding context). Stage B (warm, case has disease/species): array filters (`species n`, `disease @>`, `section_type @>`) over vector ordering. If filtered retrieval returns < 2 chunks, filters drop — **retrieval must never starve the LLM.** Retrieved chunks carry region title + page + source so every answer is citable and every bad retrieval is visible on the dashboard.

Media pipeline: evidence-first. Voice notes and photos are the farmer's real language. Media never enters the LLM raw — it is downloaded, saved to R2 as immutable evidence, then *processed into*

text/observations (transcribe with confidence; vision → structured findings), and only that enters the conversation. Two confidence rules guard safety: transcription < 0.6 → ask the farmer to confirm; vision finds nothing → request a new photo. R2 is object storage; Postgres `media` table holds references. No Pidgin TTS — true Pidgin TTS does not exist; replies are written in Pidgin-flavored English (free, human-feeling); a Nigerian-English (en-NG) TTS is parked for v2.

WhatsApp adapter: one provider now, unbreakable on swap. Baileys (unofficial, free, QR pairing) for the hackathon — but Baileys is socket-based (no HTTP webhook), re-fires events on reconnect (needs message-id dedup with TTL), and breaks/bans over time. So a **~50-line normalizer** maps any provider's event into one `InboundMessage` shape; the orchestrator never knows which provider sent it. Official WhatsApp Cloud API swaps in later by replacing one file. Farmer identity = normalized E.164 phone (+234...) → `getOrCreateFarmer`; session resume within 24h idle connects "3 more died today" to yesterday's case. Outbound goes through an **outbox queue with backoff/retry** (5s → double → max 5min, 5 attempts) and full messages only — WhatsApp cannot stream.

Observability is a product requirement, not an afterthought. Three layers, each watching a different thing: `/health` + `/ready` (liveness: process alive; readiness: DB, R2, embedding actually respond), **Sentry** (code crashes), and **Langfuse** (the AI's behavior — every turn's prompt, RAG context, tool calls, latency, tokens, cost, replayable). Every log carries `session_id` + `phone` so one farmer's journey traces across all three. Langfuse is deliberate: it turns the hardest question (*why did the AI say that?*) into a dashboard, and it is what closes the KB-growth loop when "unknown" cases get re-checked against new knowledge.

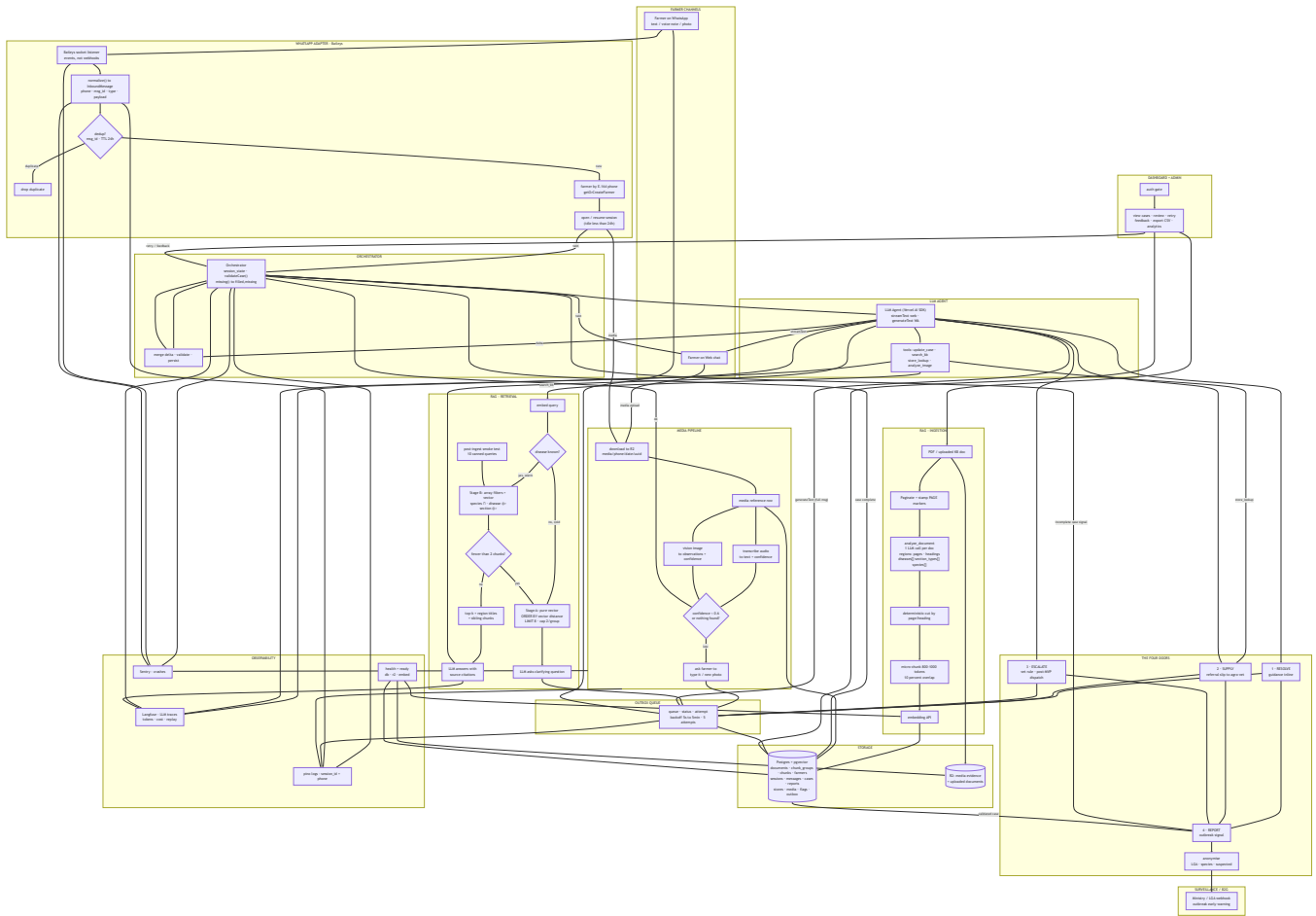
Dashboard = the growth loop. Upload documents → page-stamp → map-call → ingest. View cases, filter by critical/unknown, review, re-triage. A download button (CSV export) serves the ministry story. One addition that makes the whole RAG honest: **post-ingest smoke tests** — 5–10 canned queries per book, eyeball the top hits before it goes live.

Endpoint map (~14 core). chat (streaming, web) · whatsapp webhook (non-streaming path) · media upload · rag ingest/documents · stores list+CRUD · farmers list/profile/patch · sessions list · cases list/detail/review/retry/feedback · analytics triage + export · auth login · health · ready. Roughly a third are thin CRUD; the engineering weight sits in the orchestrator, RAG, media, adapter, and observability.

5. What we are building — the system as a whole

One conversation-first agent, running inside the channel farmers already use, that structures every interaction into a case, resolves what is safe to resolve, refers shop-appropriate cases to the nearest agro-vet with a pre-briefed slip, escalates genuinely dangerous cases to real humans, and turns every farm visit into a regional early-warning signal. The knowledge base is a growing, source-cited poultry library; the dashboard makes every decision explainable and every "unknown" a mentor; the observability stack makes every failure visible. It is **B2B-led and B2G-reinforced, farmer-free-at-point-of-use**, and intentionally narrow — poultry, middle tier, WhatsApp-first — so the skeleton under the conversation (structure → decision → handoff) is deep rather than wide.

6. System architecture — general flowchart (one diagram, whole system)



7. Stack & deployment (locked)

Full details: see `STACK.md`. Summary of every locked choice:

Piece	Runs on
Hono API · orchestrator · tools · RAG · media ingest · cases · analytics · auth	Cloudflare Workers
Dashboard + web chat UI	Cloudflare Pages (static React)
WhatsApp bridge (Baileys)	ONE Node process (fly-io/Railway/Render free)
Postgres + pgvector	Neon (free tier, 12 tables)
Media evidence + uploaded docs	Cloudflare R2
Embeddings	Google text-embedding-004 @ 768 dims (free, ~1,500 req/day)
Transcription	OpenRouter free multimodal + dynamic Pidgin prompt (fallback: whisper-1, en hint)

Piece	Runs on
Scheduled jobs	Workers Cron Triggers (surveillance digest · outbox liveness)
LLM	OpenAI GPT-4o-mini (chat · tools · vision) · pnpm monorepo TS strict · Vitest

Models & key contract numbers:

- **gpv/embed vector:** model identical at ingest & query; column frozen to `vector(768)`.
- **Pidgin transcription:** LLM-over-multimodal with a language-enforcing prompt keeps audio verbatim in Pidgin ("wetin you dey talk" stays as spoken) — bare Whisper auto-translates to English and is rejected. `conf < 0.6` → ask farmer to confirm.
- **Why a Node bridge exists at all:** Workers are stateless/short-lived; Baileys needs a persistent socket + QR/state, so it runs on a tiny host that only polls the outbox and speaks `normalize()`. Swappable with the official WhatsApp API with zero changes to the core.

Database — full ER (12 tables)

